

Chapter 1

Data Structures using C

The data structure name indicates itself that organizing the data in memory. There are many ways of organizing the data in the memory as we have already seen one of the data structures, i.e., array in C language. Array is a collection of memory elements in which data is stored sequentially, i.e., one after another. In other words, we can say that array stores the elements in a continuous manner. This organization of data is done with the help of an array of data structures. There are also other ways to organize the data in memory. Let's see the different types of data structures.

INTRODUCTION

- **OR**

- **Data Structure** is a branch of Computer Science. The study of data structure allows us to understand the organization of data and the management of the data flow in order to increase the efficiency of any process or program. Data Structure is a particular way of storing and organizing data in the memory of the computer so that these data can easily be retrieved and efficiently utilized in the future when required. The data can be managed in various ways, like the logical or mathematical model for a specific organization of data is known as a data structure.

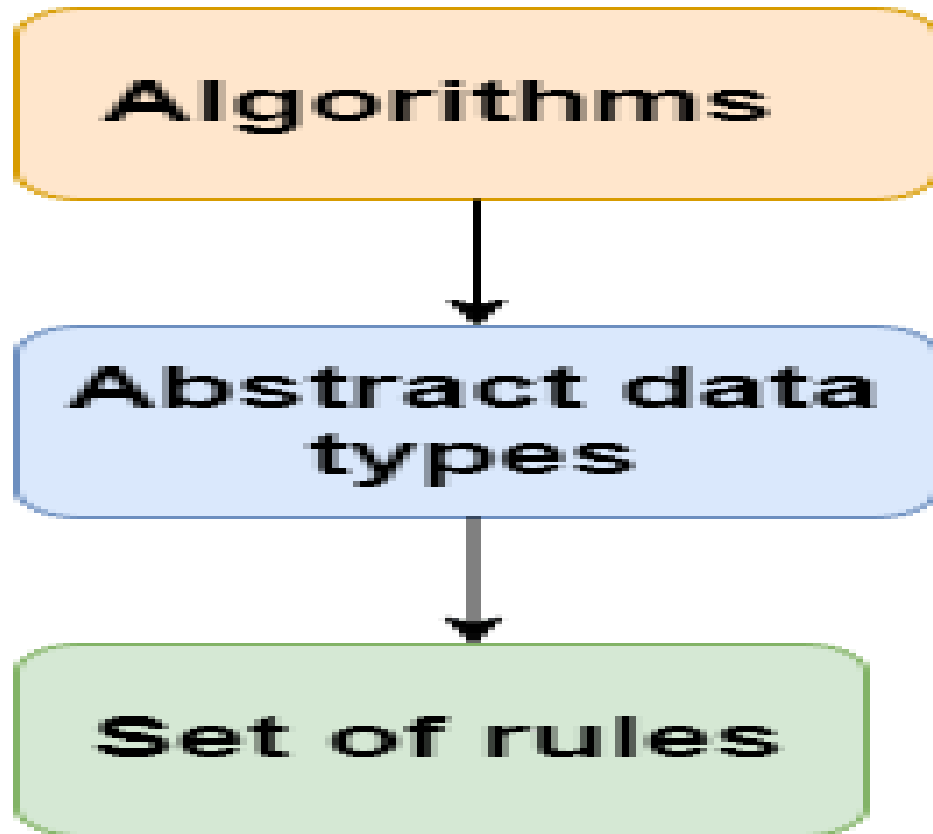
The scope of a particular data model depends on two factors:

1. First, it must be loaded enough into the structure to reflect the definite correlation of the data with a real-world object.
 2. Second, the formation should be so straightforward that one can adapt to process the data efficiently whenever necessary.
- Some examples of Data Structures are Arrays, Linked Lists, Stack, Queue, Trees, etc. Data Structures are widely used in almost every aspect of Computer Science, i.e., Compiler Design, Operating Systems, Graphics, Artificial Intelligence, and many more.

The data structure is not any programming language like C, C++, java, etc. It is a set of algorithms that we can use in any programming language to structure the data in the memory.

To structure the data in memory, 'n' number of algorithms were proposed, and all these algorithms are known as Abstract data types. These abstract data types are the set of rules.

ABSTRACT DATA TYPES



Data is a collection of facts and figures or a set of values or values of a specific format that refers to a single set of item values. The data items are then classified into sub-items, which is the group of items that are not known as the simple primary form of the item.

Basic Terminologies related to Data Structures

- Data Structures are the building blocks of any software or program. Selecting the suitable data structure for a program is an extremely challenging task for a programmer.
- The following are some fundamental terminologies used whenever the data structures are involved:
 1. **Data:** We can define data as an elementary value or a collection of values. For example, the Employee's name and ID are the data related to the Employee.
 2. **Data Items:** A Single unit of value is known as Data Item.

- 1. Group Items:** Data Items that have subordinate data items are known as Group Items. For example, an employee's name can have a first, middle, and last name.
- 2. Elementary Items:** Data Items that are unable to divide into sub-items are known as Elementary Items. For example, the ID of an Employee.

Entity and Attribute: A class of certain objects is represented by an Entity. It consists of different Attributes. Each Attribute symbolizes the specific property of that Entity.

Entity and Attribute

Attributes	ID	Name	Gender	Job Title
Values	1234	Sandhya	Female	Software Developer

Entities with similar attributes form an **Entity Set**. Each attribute of an entity set has a range of values, the set of all possible values that could be assigned to the specific attribute.

The term "information" is sometimes utilized for data with given attributes of meaningful or processed data.

1. **Field:** A single elementary unit of information symbolizing the Attribute of an Entity is known as Field.
2. **Record:** A collection of different data items are known as a Record. For example, if we talk about the employee entity, then its name, id, address, and job title can be grouped to form the record for the employee.
3. **File:** A collection of different Records of one entity type is known as a File. For example, if there are 100 employees, there will be 25 records in the related file containing data about each employee.

Understanding the Need for Data Structures

- As applications are becoming more complex and the amount of data is increasing every day, which may lead to problems with data searching, processing speed, multiple requests handling, and many more. Data Structures support different methods to organize, manage, and store data efficiently. With the help of Data Structures, we can easily traverse the data items. Data Structures provide Efficiency, Reusability, and Abstraction.

Why should we learn Data Structures?

1. Data Structures and Algorithms are two of the key aspects of Computer Science.
2. Data Structures allow us to organize and store data, whereas Algorithms allow us to process that data meaningfully.
3. Learning Data Structures and Algorithms will help us become better Programmers.
4. We will be able to write code that is more effective and reliable.
5. We will also be able to solve problems more quickly and efficiently.

Understanding the Objectives of Data Structures

- Data Structures satisfy two complementary objectives:
 1. **Correctness:** Data Structures are designed to operate correctly for all kinds of inputs based on the domain of interest. In other words, correctness forms the primary objective of Data Structure, which always depends upon the problems that the Data Structure is meant to solve.

1. Efficiency: Data Structures also requires to be efficient. It should process the data quickly without utilizing many computer resources like memory space. In a real-time state, the efficiency of a data structure is a key factor in determining the success and failure of the process.

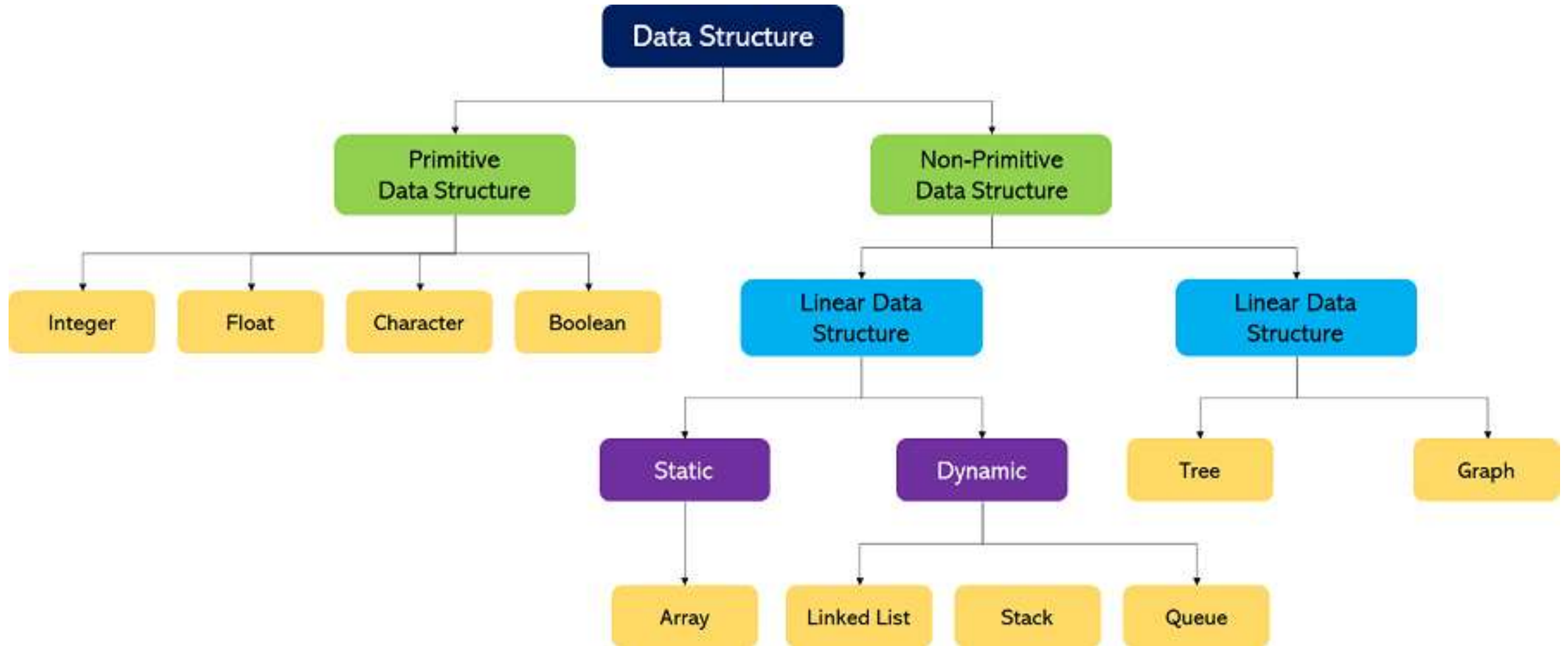
Understanding some Key Features of Data Structures

- Some of the Significant Features of Data Structures are:
1. **Robustness:** Generally, all computer programmers aim to produce software that yields correct output for every possible input, along with efficient execution on all hardware platforms. This type of robust software must manage both valid and invalid inputs.

1. Adaptability: Building software applications like web Browsers, Word Processors, and Internet Search Engine include huge software systems that require correct and efficient working or execution for many years. Moreover, software evolves due to emerging technologies or ever-changing market conditions.

Reusability: The features like Reusability and Adaptability go hand in hand. It is known that the programmer needs many resources to build any software, making it a costly enterprise. However, if the software is developed in a reusable and adaptable way, then it can be applied in most future applications. Thus, by executing quality data structures, it is possible to build reusable software, which appears to be cost-effective and timesaving.

Classification of Data Structures



- A Data Structure delivers a structured set of variables related to each other in various ways. It forms the basis of a programming tool that signifies the relationship between the data elements and allows programmers to process the data efficiently.
- We can classify Data Structures into two categories:
 1. Primitive Data Structure
 2. Non-Primitive Data Structure
- Primitive Data Structures

Classification of Data Structures

1. **Primitive Data Structures** are the data structures consisting of the numbers and the characters that come **in-built** into programs.
2. These data structures can be manipulated or operated directly by machine-level instructions.
3. Basic data types like **Integer**, **Float**, **Character**, and **Boolean** come under the Primitive Data Structures.
4. These data types are also called **Simple data types**, as they contain characters that can't be divided further

Non-Primitive Data Structures

1. **Non-Primitive Data Structures** are those data structures derived from Primitive Data Structures.
2. These data structures can't be manipulated or operated directly by machine-level instructions.

1. The focus of these data structures is on forming a set of data elements that is either **homogeneous** (same data type) or **heterogeneous** (different data types).
2. Based on the structure and arrangement of data, we can divide these data structures into two sub-categories -
 - 1.Linear Data Structures
 - 2.Non-Linear Data Structures

Linear Data Structures

- A data structure that preserves a linear connection among its data elements is known as a Linear Data Structure. The arrangement of the data is done linearly, where each element consists of the successors and predecessors except the first and the last data element. However, it is not necessarily true in the case of memory, as the arrangement may not be sequential.
- Based on memory allocation, the Linear Data Structures are further classified into two types:
 1. **Static Data Structures:** The data structures having a fixed size are known as Static Data Structures. The memory for these data structures is allocated at the compiler time, and their size cannot be changed by the user after being compiled; however, the data stored in them can be altered.

1. The **Array** is the best example of the Static Data Structure as they have a fixed size, and its data can be modified later.
2. **Dynamic Data Structures:** The data structures having a dynamic size are known as Dynamic Data Structures. The memory of these data structures is allocated at the run time, and their size varies during the run time of the code. Moreover, the user can change the size as well as the data elements stored in these data structures at the run time of the code.

Linked Lists, Stacks, and Queues are common examples of dynamic data structures

Non-Linear Data Structure

- A form of data structure where the data elements don't stay arranged linearly or sequentially.
- Trees and Graphs are the types of non-linear data structures.
- The nodes in the tree data structure are arranged in hierarchical order.
- In A graph is with a finite number of vertices and edges, and these edges are used to connect the vertices.

Data Structures Operations

- The major or the common operations that can be performed on the data structures are:
 - **Searching:** We can search for any element in a data structure.
 - **Sorting:** We can sort the elements of a data structure either in an ascending or descending order.
 - **Insertion:** We can also insert the new element in a data structure.
 - **Updation:** We can also update the element, i.e., we can replace the element with another element.
 - **Deletion:** We can also perform the delete operation to remove the element from the data structure.

Advantages of Data structures

- **The following are the advantages of a data structure:**
 - **Efficiency:** If the choice of a data structure for implementing a particular ADT is proper, it makes the program very efficient in terms of time and space.
 - **Reusability:** The data structure provides reusability means that multiple client programs can use the data structure.
 - **Abstraction:** The data structure specified by an ADT also provides the level of abstraction. The client cannot see the internal working of the data structure, so it does not have to worry about the implementation part. The client can only see the interface.

Data Structures Algorithm

- An algorithm is a process or a set of rules required to perform calculations or some other problem-solving operations especially by a computer. The formal definition of an algorithm is that it contains the finite set of instructions which are being carried in a specific order to perform the specific task. It is not the complete program or code; it is just a solution (logic) of a problem, which can be represented either as an informal description using a Flowchart or Pseudo code.

We need algorithms because of the following reasons:

- **Scalability:** It helps us to understand the scalability. When we have a big real-world problem, we need to scale it down into small-small steps to easily analyze the problem.

Performance: The real-world is not easily broken down into smaller steps. If the problem can be easily broken into smaller steps means that the problem is feasible.

Example: write an algorithm to add two numbers entered by the user

•The following are the steps required to add two numbers entered by the user:

- Step 1: Start
- Step 2: Declare three variables a, b, and sum.
- Step 3: Enter the values of a and b.
- Step 4: Add the values of a and b and store the result in the sum variable, i.e., $\text{sum} = a + b$.
- Step 5: Print sum
- Step 6: Stop

Characteristics of an Algorithm

•The following are the characteristics of an algorithm:

- **Input:** An algorithm has some input values. We can pass 0 or some input value to an algorithm.
- **Output:** We will get 1 or more output at the end of an algorithm.
- **Unambiguity:** An algorithm should be unambiguous which means that the instructions in an algorithm should be clear and simple.

- **Finiteness:** An algorithm should have finiteness. Here, finiteness means that the algorithm should contain a limited number of instructions, i.e., the instructions should be countable.
- **Effectiveness:** An algorithm should be effective as each instruction in an algorithm affects the overall process.
- **Language independent:** An algorithm must be language-independent so that the instructions in an algorithm can be implemented in any of the languages with the same output.

Dataflow of an Algorithm

Problem: A problem can be a real-world problem or any instance from the real-world problem for which we need to create a program or the set of instructions. The set of instructions is known as an algorithm.

Algorithm: An algorithm will be designed for a problem which is a step by step procedure.

Input: After designing an algorithm, the required and the desired inputs are provided to the algorithm.

Processing unit: The input will be given to the processing unit, and the processing unit will produce the desired output.

Output: The output is the outcome or the result of the program.

Factors of an Algorithm

- **The following are the factors that we need to consider for designing an algorithm:**
 - **Modularity:** If any problem is given and we can break that problem into small-small modules or small-small steps, which is a basic definition of an algorithm, it means that this feature has been perfectly designed for the algorithm.
 - **Correctness:** The correctness of an algorithm is defined as when the given inputs produce the desired output, which means that the algorithm has been designed algorithm. The analysis of an algorithm has been done correctly.
 - **Maintainability:** Here, maintainability means that the algorithm should be designed in a very simple structured way so that when we redefine the algorithm, no major change will be done in the algorithm.

- **Functionality:** It considers various logical steps to solve the real-world problem.
- **Robustness:** Robustness means that how an algorithm can clearly define our problem.
- **User-friendly:** If the algorithm is not user-friendly, then the designer will not be able to explain it to the programmer.
- **Simplicity:** If the algorithm is simple then it is easy to understand.
- **Extensibility:** If any other algorithm designer or programmer wants to use your algorithm then it should be extensible.

Importance of Algorithms

1. **Theoretical importance:** When any real-world problem is given to us and we break the problem into small-small modules. To break down the problem, we should know all the theoretical aspects.
2. **Practical importance:** As we know that theory cannot be completed without the practical implementation. So, the importance of algorithm can be considered as both theoretical and practical.

Issues of Algorithms

The following are the issues that come while designing an algorithm:

- **How to design algorithms:** As we know that an algorithm is a step-by-step procedure so we must follow some steps to design an algorithm.
- **How to analyze algorithm efficiency**

Approaches of Algorithm

- **The following are the approaches used after considering both the theoretical and practical importance of designing an algorithm:**
 - **Brute force algorithm:** The general logic structure is applied to design an algorithm. It is also known as an exhaustive search algorithm that searches all the possibilities to provide the required solution. Such algorithms are of two types:
 - **Optimizing:** Finding all the solutions of a problem and then take out the best solution or if the value of the best solution is known then it will terminate if the best solution is known.
 - **Sacrificing:** As soon as the best solution is found, then it will stop.

- **Divide and conquer:** It is a very implementation of an algorithm. It allows you to design an algorithm in a step-by-step variation. It breaks down the algorithm to solve the problem in different methods. It allows you to break down the problem into different methods, and valid output is produced for the valid input. This valid output is passed to some other function.

- **Greedy algorithm:** It is an algorithm paradigm that makes an optimal choice on each iteration with the hope of getting the best solution. It is easy to implement and has a faster execution time. But, there are very rare cases in which it provides the optimal solution.
- **Dynamic programming:** It makes the algorithm more efficient by storing the intermediate results. It follows five different steps to find the optimal solution for the problem:

- 1.It breaks down the problem into a subproblem to find the optimal solution.
- 2.After breaking down the problem, it finds the optimal solution out of these subproblems.
- 3.Stores the result of the subproblems is known as memorization.
- 4.Reuse the result so that it cannot be recomputed for the same subproblems.
- 5.Finally, it computes the result of the complex program.

- **Branch and Bound Algorithm:** The branch and bound algorithm can be applied to only integer programming problems. This approach divides all the sets of feasible solutions into smaller subsets. These subsets are further evaluated to find the best solution.
- **Randomized Algorithm:** As we have seen in a regular algorithm, we have predefined input and required output. Those algorithms that have some defined set of inputs and required output, and follow some described steps are known as deterministic algorithms.

- What happens that when the random variable is introduced in the randomized algorithm?. In a randomized algorithm, some random bits are introduced by the algorithm and added in the input to produce the output, which is random in nature. Randomized algorithms are simpler and efficient than the deterministic algorithm.
- **Backtracking:** Backtracking is an algorithmic technique that solves the problem recursively and removes the solution if it does not satisfy the constraints of a problem.

The major categories of algorithms are given below:

- **Sort:** Algorithm developed for sorting the items in a certain order.
- **Search:** Algorithm developed for searching the items inside a data structure.
- **Delete:** Algorithm developed for deleting the existing element from the data structure.
- **Insert:** Algorithm developed for inserting an item inside a data structure.
- **Update:** Algorithm developed for updating the existing element inside a data structure.

Algorithm Complexity

- The performance of the algorithm can be measured in two factors:
 - **Time complexity:** The time complexity of an algorithm is the amount of time required to complete the execution. The time complexity of an algorithm is denoted by the big O notation. Here, big O notation is the asymptotic notation to represent the time complexity. The time complexity is mainly calculated by counting the number of steps to finish the execution. Let's understand the time complexity through an example.
 - **Space complexity:** An algorithm's space complexity is the amount of space required to solve a problem and produce an output. Similar to the time complexity, space complexity is also expressed in big O notation.

For an algorithm, the space is required for the following purposes:

1. To store program instructions
2. To store constant values
3. To store variable values
4. To track the function calls, jumping statements, etc.

Types of Algorithms

- **The following are the types of algorithm:**

- **Search Algorithm**
- **Sort Algorithm**

- **Search Algorithm**

- On each day, we search for something in our day to day life. Similarly, with the case of computer, huge data is stored in a computer that whenever the user asks for any data then the computer searches for that data in the memory and provides that data to the user.

Sorting Algorithms

Sorting algorithms are used to rearrange the elements in an array or a given data structure either in an ascending or descending order. The comparison operator decides the new order of the elements.

THANK YOU